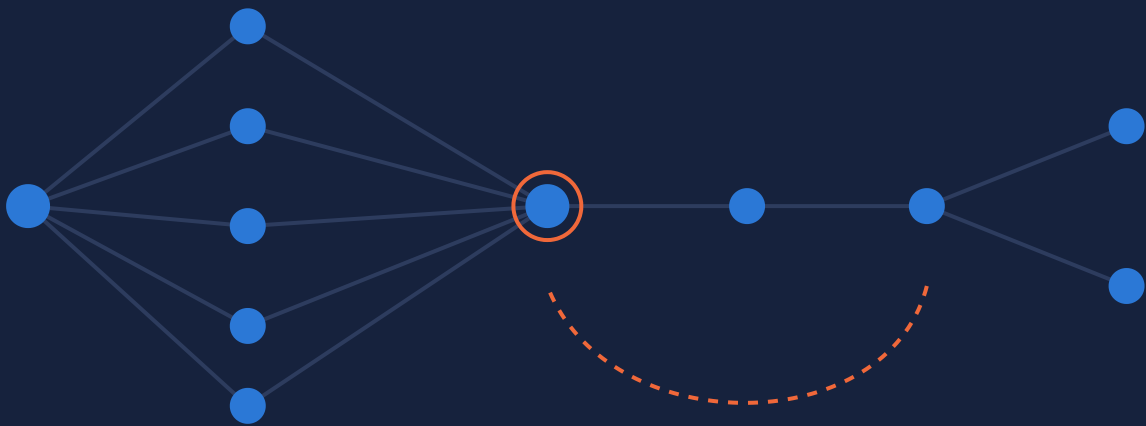


ORCA

A beginner's guide to agentic marine intelligence:
designing specialised agents and orchestrating
them with LangGraph



What the project does, how its thirteen agents were designed,
how LangGraph runs them, and why each part exists —
explained from zero, no AI or ocean-science background needed.

Smart India Hackathon · problem statement SIH26176 · September 2026

Contents

How to read this guide	3
Part 1 — Foundations from zero	4
1. The problem ORCA solves	5
2. Ocean data in ten minutes	5
3. What an AI agent is	6
4. From one agent to many: the blackboard	7
5. LangGraph from zero	8
Part 2 — Designing the agents	10
6. Seven design principles	11
7. The agent contract	11
8. The thirteen agents	13
9. The planner: from a sentence to a task graph	15
10. Re-planning: agents that change their mind	16
11. Evidence, confidence and conflicts	17
12. Designing for failure	18
Part 3 — Orchestrating with LangGraph	19
13. Why LangGraph, and what it replaced	20
14. The graph state and the runtime context	20
15. The nodes	21
16. Dynamic fan-out with Send	22
17. Conditional edges and the re-plan loop	23
18. One real run, superstep by superstep	25
19. Streaming the graph to the screen	26
20. Testing the graph	27
Part 4 — The rest of the system	29
21. The big picture	30
22. Data connectors and the three modes	30
23. The deterministic core	31
24. Where the LLM is allowed, and its guard	32
25. The API and the mission-control console	32
26. Evaluation: how we know it works	33
27. How it was built	34
Part 5 — Using it, and reference	36
28. Setting it up	37
29. A guided demonstration	37
30. Limitations	38
31. Future improvements	39
32. Glossary	39
33. Code map and command cheat sheet	40
34. What you now understand	41

How to read this guide

This guide explains ORCA, a working prototype that answers questions about the sea around India. For example: "Is it safe for a fishing boat to leave Kochi tomorrow at 5 AM?" It answers by sending thirteen specialised software agents to gather official ocean and weather data, check each other's work, and agree on a decision whose evidence you can inspect. You don't need any background in artificial intelligence or oceanography to follow it.

Most of the guide is about the **agents**: how each one was designed, how they share information without talking to each other in free text, and how **LangGraph** (an open-source library for building agent workflows as graphs) schedules them, runs them in parallel and loops back when the evidence is not good enough.

The guide has five parts:

- **Part 1 — Foundations from zero.** The problem, the minimum you need to know about ocean data, what an agent is, why many agents need a shared "blackboard", and the LangGraph vocabulary.
- **Part 2 — Designing the agents.** The design principles, the contract every agent obeys, all thirteen agents one by one, the planner, re-planning, evidence and failure handling.
- **Part 3 — Orchestrating with LangGraph.** The real graph in `apps/api/orca/graph.py`: state, nodes, `Send` fan-out, conditional edges, and one real run traced superstep by superstep.
- **Part 4 — The rest of the system.** Data connectors, the three data modes, the deterministic risk and confidence models, the optional LLM, the API, the map console, evaluation, and how the project was built.
- **Part 5 — Using it, and reference.** Setup, a guided demo, limitations, future work, a glossary and a code map.

Throughout the guide you'll see coloured boxes:

Everyday analogy

Compares a technical idea to something from everyday life.

Why this matters

Explains what would go wrong without this idea.

In ORCA

Points to where the idea lives in the ORCA code.

Watch out

A common mistake or a limitation to keep in mind.

Key idea

The one sentence to remember from a section.

Code snippets are short excerpts from the real project files; the file path is printed above each one. Numbers from example runs come from ORCA's deterministic DEMO mode, so you get the same decisions if you re-run them. Timings will differ slightly on your machine.

PART 1

Foundations from zero

Before looking at any code we need four ideas: what problem ORCA solves, what ocean data looks like, what an "agent" is, and how a group of agents can cooperate. The last section introduces LangGraph, the library that runs ORCA's agents.

-
1. The problem ORCA solves
 2. Ocean data in ten minutes
 3. What an AI agent is
 4. From one agent to many: the blackboard
 5. LangGraph from zero
-

1. The problem ORCA solves

India has more than 11,000 km of coastline and several million people who depend on fishing. Every morning, a fisherman in Kochi or Visakhapatnam has to decide whether it is safe to go out. A coast-guard officer has to decide whether to warn a district. A researcher wants to know why chlorophyll fell off Kerala last fortnight.

The information to answer these questions exists, but it is scattered:

Information	Who publishes it
Wave and swell forecasts	Several global ocean models (Météo-France MFWAM, ECMWF WAM, NOAA GFS-Wave, DWD GWAM), each slightly different
Wind, gusts, rain, thunderstorm potential	Weather models (ECMWF IFS, NOAA GFS, DWD ICON, UK Met Office)
Sea temperature, chlorophyll from space	Satellites, via NOAA CoastWatch and NASA
Cyclone tracks	GDACS (a UN / European Commission alert system)
Official Indian warnings	IMD (weather), INCOIS (ocean state, fishing zones). Not openly machine-readable today.
Where you may and may not go	Exclusive economic zones, marine protected areas, international boundaries

Problem statement **SIH26176** asks for an "agentic" system: software that plans its own work, fetches real data, reasons over it and explains itself. ORCA is our answer. You ask a question in English or one of seven other Indian languages. ORCA gives a decision (**GO**, **CAUTION**, **DON'T GO** or **NO RELIABLE ASSESSMENT**) with the reasons, the sources, a confidence number, a map, and a full trace of what every agent did.

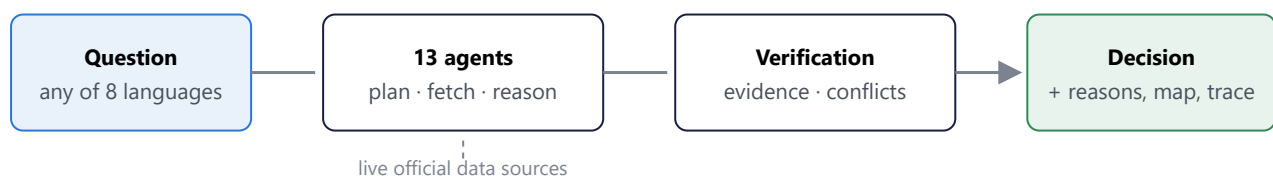


Figure 1 — What ORCA does, in one line.

Why this matters

A wrong "it's safe" can cost a life. So ORCA must never invent a number, never present simulated data as real, and never say "conditions appear safe" when the data needed to judge is missing. Most design decisions in this guide follow from that one requirement.

2. Ocean data in ten minutes

You need five ideas about ocean data to follow the rest of the guide.

- 1. Forecasts come from models.** A forecast model is a large physics simulation run by a national weather agency several times a day. Each run produces values on a grid (for example every 0.25° , about 28 km) for every hour of the next few days. Different agencies' models disagree, sometimes a lot.
- 2. Observations come from satellites and are late.** A satellite map of sea-surface temperature (SST) or chlorophyll describes the past, usually one or two days ago, and has gaps under clouds.
- 3. Every number has a place and a time.** "2.4 m waves" means nothing without *where* (which grid cell, how far from the boat) and *when* (which hour, from which model run). ORCA keeps both for every value.
- 4. Data goes stale.** A forecast from a model run 30 hours ago is worse than one from 3 hours ago. ORCA labels each datum as **LIVE**, **RECENT**, **STALE** or **UNAVAILABLE**, and labels recorded or synthetic data as **REPLAY** or **SIMULATED**, never as **LIVE**.
- 5. Safety thresholds depend on the boat.** 2 m waves are routine for a cargo ship and dangerous for a small fishing boat. ORCA's risk model has separate thresholds for small craft, mechanised boats and large vessels.

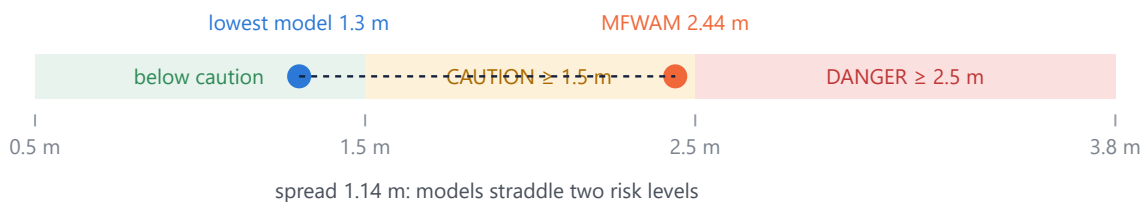


Figure 2 — Wave models disagreeing 25 km off Chennai (DEMO scenario conflicting_sources, small-craft thresholds).

Everyday analogy

Asking four weather apps and getting four answers is normal. A careful person doesn't average them. They take the worst plausible one seriously and note that the apps disagreed. ORCA does exactly that: the worst-case value decides, and the disagreement lowers the confidence and triggers a tie-breaker.

3. What an AI agent is

A **large language model** (LLM) such as Claude reads text and writes text. On its own it can't look anything up, and it sometimes states numbers that are fluent but wrong (a "hallucination"). An **agent** is a program that wraps a decision-maker in a loop with **tools**: it decides what to do next, calls a tool (an API, a calculation), looks at the result, and repeats until it can answer.

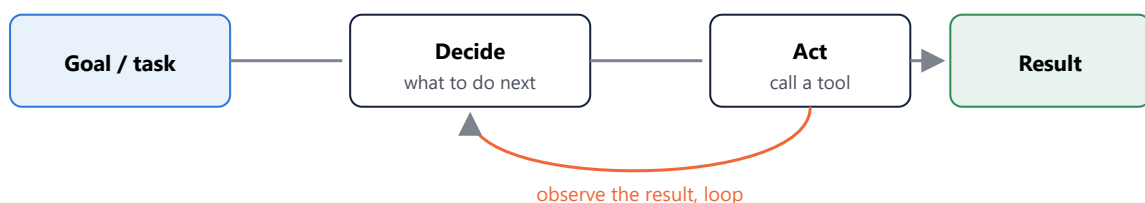


Figure 3 — The agent loop: decide, act with a tool, observe, repeat.

Here is the most important design choice in ORCA: **the "decide" step does not have to be an LLM**. In ORCA, eleven of the thirteen agents decide with deterministic code (explicit rules, formulas and geometry), because a safety decision has to be reproducible and explainable. An LLM is allowed in exactly two places: understanding an unusual question (only when the rule-based parser is unsure), and rephrasing an answer that has already been computed. Even then a guard checks every number (section 24).

Key idea

"Agentic" describes the structure: planning, tool use, self-checking and re-planning. It doesn't require an LLM in every step. ORCA is agentic in structure and deterministic where safety is at stake.

4. From one agent to many: the blackboard

One agent that does everything (parsing, geography, ocean data, weather, satellites, risk, routing, translation) would be a huge and fragile program. So ORCA uses **specialised agents**, each an expert in one thing. That raises a question: how do experts cooperate?

The simplest answer is to let them chat: the ocean agent writes a paragraph, the risk agent reads it. That breaks immediately. Numbers get rounded, units get lost, and nobody can check which paragraph a decision came from. ORCA uses the classic **blackboard** pattern instead:

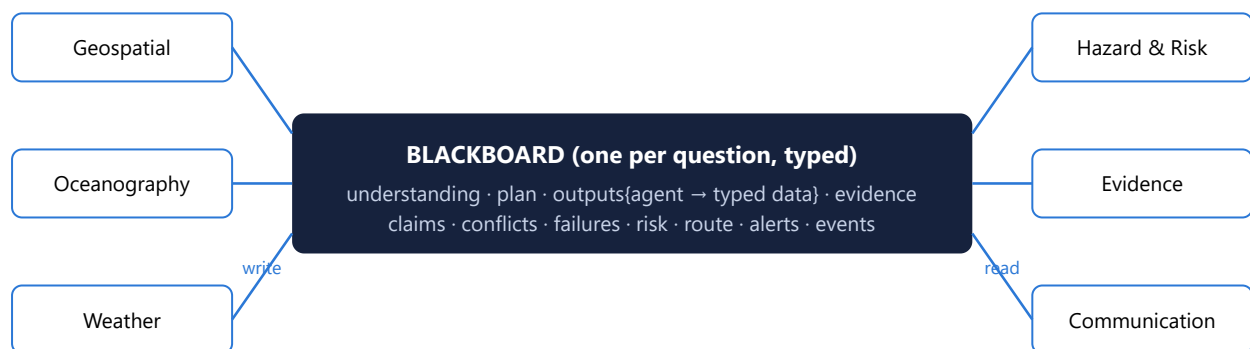


Figure 4 — Agents never talk to each other. They read and write typed data on a shared blackboard.

Everyday analogy

Think of a hospital patient chart. The radiologist doesn't phone the surgeon with a story. They write the scan result in a fixed field of the chart, and the surgeon reads it there. Anyone can later audit who wrote what and when.

The blackboard is a Pydantic model (Pydantic is a Python library for typed, validated data). Every agent's output is also a Pydantic model with a published JSON schema, so "what the ocean agent said" is a structured object with units, sources and timestamps, not a paragraph.

In ORCA

The blackboard is `Blackboard` in `apps/api/orca/core/schemas/blackboard.py`. The per-query wrapper that agents receive is `RunState` in `apps/api/orca/agents/base.py`. The typed output schemas of all agents are served live at `GET /api/agents`.

The blackboard settles *how* agents share data. We still need something that decides *who runs when*: in which order, which in parallel, and what happens when one fails or the evidence is weak. That is the **orchestrator**, and in ORCA it is built with LangGraph.

5. LangGraph from zero

LangGraph is an open-source Python library for building agent workflows as **graphs**. ORCA uses version 1.2. You need eight words:

Word	Meaning
State	A typed dictionary that flows through the graph. Every node receives it and returns an update to it.
Node	A function (usually async) that does one step of work. In ORCA each agent is a node.
Edge	"After node A, run node B."
Conditional edge	A small routing function that looks at the state and returns where to go next. This is how graphs make decisions and loop.
Reducer	A rule for merging updates to the same state key, e.g. <code>operator.add</code> concatenates lists. Needed when parallel nodes write the same key.
Send	A special return value of a routing function: "run node X with this payload". Returning several <code>Send</code> s fans out work in parallel, even to the same node several times.
Superstep	One tick of the graph. All nodes scheduled for the tick run concurrently, and the next tick starts only when all of them have finished.
Runtime context	Per-run objects (connections, clocks, big data) passed to every node alongside the state, but not part of it.

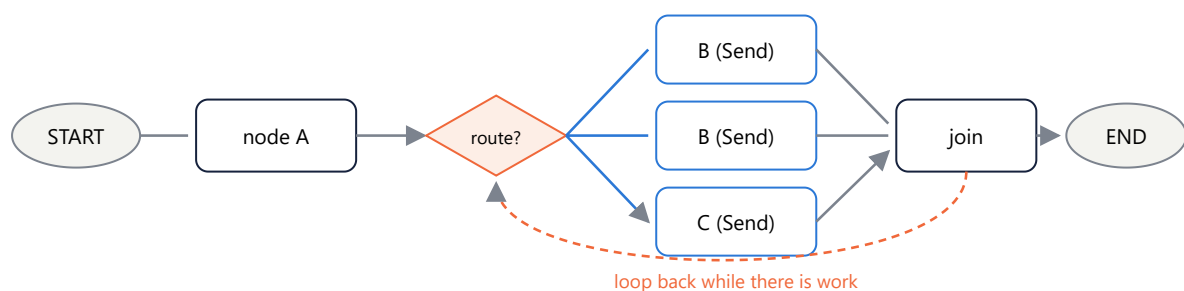


Figure 5 — The LangGraph vocabulary in one picture: a conditional edge fans out three Sends in one superstep; join waits for all of them.

You write the graph once (add nodes, add edges), call `compile()`, and then invoke the compiled graph as many times as you like. LangGraph handles the scheduling, parallelism, merging of state updates, and loop limits (the "recursion limit").

Why this matters

Without a graph library you write the scheduler yourself: find ready tasks, start them together, wait, merge results, check for loops, guard against infinite re-planning. ORCA's first version did exactly that by hand (section 13). A graph library turns that code into a declared structure that can be drawn, tested and reasoned about.

PART 2

Designing the agents

This is the heart of ORCA. Each agent was designed as a small expert with a fixed contract: what it reads, which tools it may use, what typed result it must return, and how it fails. This part explains those decisions agent by agent.

6. Seven design principles

7. The agent contract

8. The thirteen agents

9. The planner: from a sentence to a task graph

10. Re-planning: agents that change their mind

11. Evidence, confidence and conflicts

12. Designing for failure

6. Seven design principles

Before writing any agent we fixed seven rules. Every agent in ORCA follows all of them, and most of the bugs found during development were violations of one of them.

#	Principle	What it means in practice
1	One responsibility per agent	The weather agent fetches weather. It doesn't judge risk. The hazard agent judges risk. It doesn't fetch anything.
2	Typed outputs, never free text	Every agent returns a Pydantic model. Downstream agents read fields, not sentences.
3	Deterministic where it matters	Risk, confidence, geometry, routing and statistics are formulas with published parameters. Same input, same answer.
4	The LLM stays at the edges	Only interpretation fallback and final wording. Its output is validated against the gazetteer or checked number by number.
5	Every claim carries evidence	Every value keeps its source, product time, retrieval time, location and freshness, all the way to the screen.
6	Failure is a result, not a crash	A failed source or agent is recorded and reported. The pipeline continues and says what is missing.
7	Same agents in every mode	LIVE, REPLAY and DEMO differ only in where bytes come from. The agents can't tell and don't need to.

Key idea

An agent is not "a prompt". In ORCA an agent is a Python class with a name, a declared list of tools, a declared list of inputs it consumes, a typed output model, a timeout, and one `run()` method.

7. The agent contract

All agents inherit from one abstract base class. This is the whole contract:

`apps/api/orca/agents/base.py`

```
class Agent(ABC):
    name: ClassVar[str]
    title: ClassVar[str]
    responsibility: ClassVar[str]
    tools: ClassVar[list[str]]
    consumes: ClassVar[list[str]] = []
    output_model: ClassVar[Optional[type[BaseModel]]] = None
    deterministic: ClassVar[bool] = True # False only where an LLM may participate
    timeout_s: ClassVar[float] = 90.0

    @abstractmethod
    async def run(self, task: PlanTask, st: RunState) -> AgentOutcome: ...
```

And this is what every `run()` must return:

apps/api/orca/agents/base.py

```
@dataclass
class AgentOutcome:
    status: AgentStatus          # SUCCEEDED | PARTIAL | FAILED
    summary: str
    output: Optional[BaseModel] = None  # serialisable typed output (goes on the blackboard)
    typed: Any = None              # in-memory rich objects for downstream agents
    confidence: Optional[float] = None
    sources: list[str] = field(default_factory=list)
    tools: list[str] = field(default_factory=list)
    failures: list[SourceFailure] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)
```

Three details in this contract are deliberate:

1. **Two kinds of output.** `output` is a small, JSON-serialisable summary that is stored and shown in the UI. `typed` is for large in-memory objects (full hourly series for every model at every sample point, forecast grids) that downstream agents need but that would bloat the stored trace.
2. **PARTIAL is a real status.** If two of three wave models answer, the ocean agent succeeds *partially*. The distinction feeds the confidence model and the "excluded sources" list.
3. **Failures are data.** A `SourceFailure` records which source failed, how (timeout, HTTP 404, not configured, IP not whitelisted) and when. They are merged into `blackboard.failures` and shown to the user.

Here is how a concrete agent fills in the declarations:

apps/api/orca/agents/ocean.py

```
class OceanAgent(Agent):
    name = "ocean"
    title = "Oceanography Agent"
    responsibility = "Waves, swell, currents and model SST from multiple independent ocean models, ..."
    tools = ["om_mfwam.points", "om_ecmwf_wam.points", "om_gfs_wave.points", "om_smoc.points",
            "om_dwd_gwam.points (tie-breaker)", "openmeteo.run_info"]
    consumes = ["geospatial"]
    output_model = RetrievalOutput

    async def run(self, task, st):
        geo = st.typed.get("geospatial")          # read upstream typed output
        tiebreak = task.params.get("tiebreaker", False)  # set by re-planning rule RP1/RP2
        wave_models = reg.tiebreaker("marine") if tiebreak else reg.wave_models()
        ...
```

In ORCA

`consumes` is not just documentation. The planner uses the same dependency names to build the task graph, the Agents page draws them as edges, and a test checks that agents only run after what they consume.

Watch out

An early version of the geospatial agent returned its typed output but forgot to put it in `st.typed`, so every downstream agent silently saw "no sample points". The fix was one line (`st.typed[self.name] = out`). The lesson: the contract must be checked by tests, not by memory.

8. The thirteen agents

The agents fall into four layers: understanding and planning, retrieval, reasoning, and output.

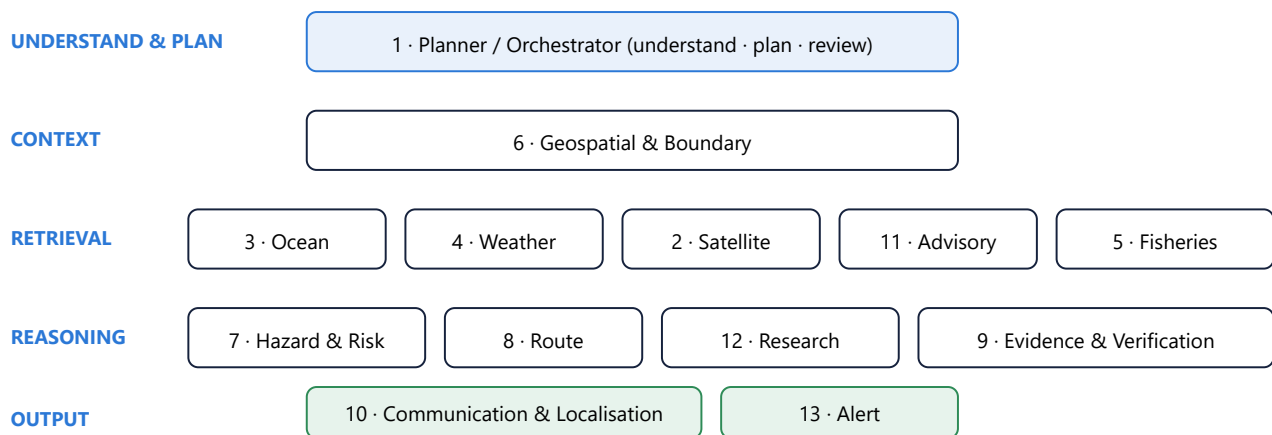


Figure 6 — The thirteen agents in four layers. The numbers are ORCA's official agent numbers.

Understanding and planning

1 · Planner / Orchestrator. Turns the question into an `Understanding` (intent, places, exact time window in IST and UTC, vessel class, language, role) and an `ExecutionPlan`, a list of tasks with dependencies. After each round it reviews the results and may add tasks (section 10). It is the only agent that appears as three graph nodes (`understand`, `plan`, `review`) because it acts at three moments.

Context

6 · Geospatial & Boundary. Converts "25 km off Chennai" into real coordinates at sea. It projects seaward along the bearing that gets away from the coast fastest (among 72 candidates) and designs the *sampling*: an assessment point, transit points from the harbour, and a small ring, so that the answer covers the whole trip, not one dot. It also finds the exclusive economic zone, distance to coast, depth, distance to the international boundary, and any marine protected area, restricted zone, seasonal fishing ban or user geofence. Everything is computed with shapely and pyproj in a local metric projection, so distances are true kilometres.

Retrieval

3 · Oceanography. Waves, swell, wave period and currents from three independent wave models (MFWAM, ECMWF WAM, GFS-Wave) plus the SMOC current model, at every sample point for every hour. It works in point mode (safety questions) or grid mode (routing). With the `tiebreaker` flag it queries DWD GWAM instead.

4 • Weather. Wind, gusts, rain, pressure, CAPE (an index of thunderstorm energy) and visibility from ECMWF IFS, GFS and ICON. Its tie-breaker is the UK Met Office model. It also has a history mode that returns past weeks of wind for research questions.

2 • Satellite / Earth Observation. Satellite SST, SST anomaly and chlorophyll from NOAA CoastWatch ERDDAP, with product dates, latency and resolution. It serves two purposes: context and a cross-check of the models' SST. For research it builds daily area-mean time series.

11 • Advisory. Tropical cyclones from GDACS (track, wind radii, forecast cone), and the IMD and INCOIS warning adapters. It computes the distance from every sample point to every active storm and whether a point lies inside a wind radius.

5 • Fisheries. ORCA's productivity indicator: where sharp SST fronts and elevated chlorophyll coincide, clustered into candidate zones with depth. It is always labelled probabilistic ("presence of fish is not implied") and kept separate from official INCOIS Potential Fishing Zone advisories.

Reasoning

7 • Hazard & Risk. Collects every source's samples for every factor (waves, wind, gusts, current, rain, thunderstorm potential, visibility, cyclone distance, boundary distance, official warnings) and applies the deterministic risk model `orca-risk-1.0`. It produces an hourly timeline, the onset time of caution, and per-station results for regional questions. It never fetches data itself.

8 • Route. Builds an hourly risk "cube" (the worst of all models at each grid cell and hour) and runs a time-dependent A* search. The cost of a step depends on the risk *at the time you would be there*. It returns three options (shortest, recommended, safest), smooths them without increasing risk, and evaluates every kilometre at ETA, including exactly where a route would enter a protected area.

12 • Research. Compares periods (Welch t-test), fits trends, finds lagged correlations and evaluates four rule-based hypotheses, each with supporting and contradicting evidence. It labels every statement as OBSERVATION, CORRELATION, HYPOTHESIS or CONCLUSION, and refuses to claim causation.

9 • Evidence & Verification. The agents' referee. It turns results into **claims**, attaches **evidence items** with weights, detects **conflicts** between sources, computes **confidence**, names the weakest link and assembles the final decision package with recommendations and caveats (section 11).

Output

10 • Communication & Localisation. Renders the same verified result for four roles (fisherman, authority, researcher, operator) in English and seven Indian languages (Hindi, Marathi, Tamil, Telugu, Malayalam, Kannada, Bengali). It uses templates. Optional LLM narration is accepted only if every number in it can be found in the computed facts.

13 • Alert. Turns hazards into alerts using a four-step pipeline: EVENT → VERIFY → GENERATE → DELIVER. An alert is marked *verified* only if an official advisory supports it, or at least two independent sources agree with confidence ≥ 0.5 . Alerts are de-duplicated and pushed live to the console. A background monitor runs the same agents on a schedule.

Agent	Consumes	Typed output	LLM?
Planner	request, conversation	Understanding , ExecutionPlan	fallback only
Geospatial	understanding	GeoOutput	no
Ocean · Weather	geospatial	RetrievalOutput	no
Satellite	geospatial	SatelliteOutput	no
Advisory	geospatial	AdvisoryOutput	no
Fisheries	geospatial	FisheriesOutput	no
Hazard	ocean, weather, advisory, geospatial, route	HazardOutput	no
Route	geospatial, ocean, weather	RouteOutput	no
Research	satellite, weather	ResearchOutput	no
Evidence	all reasoning + retrieval outputs	FinalAssessment , claims, evidence	no
Communication	evidence, hazard, route, fisheries, research	LocalizedResponse	optional, guarded
Alert	evidence, hazard, route	Alert s	no

9. The planner: from a sentence to a task graph

The planner works in two steps.

Step 1: understand. A deterministic parser does the work. It detects the language from the Unicode script (with marker words to tell Hindi from Marathi) and matches the intent against a multilingual cue lexicon in a fixed priority order. It finds places in a 64-place gazetteer that includes local-language names and Indic case endings, so "ராமேஸ்வரத்தில்" resolves to Rameswaram. It also turns time phrases into an explicit window. "Tomorrow morning" becomes *tomorrow 05:00–11:00 IST*, and the assumption is written down. Only if the parser's confidence is below 0.5 is the optional LLM asked. Even then it must answer with gazetteer identifiers, which are validated.

Step 2: plan. Each intent has a plan template: a small dependency graph of tasks. Here is the safety-check template, straight from the code:

apps/api/orca/agents/planner.py

```

T("geospatial", "geospatial", reason="assessment point, trip transect, zone & boundary context",
critical=True, ...)
T("ocean", "ocean", ["geospatial"], "waves, swell, currents from 3 wave models + SMOC", True)
T("weather", "weather", ["geospatial"], "wind, gusts, rain, lightning potential from 3 NWP models",
True)
T("satellite", "satellite", ["geospatial"], "satellite SST / anomaly / chlorophyll (context & cross-
check)")
T("advisory", "advisory", ["geospatial"], "cyclones (GDACS), IMD & INCOIS warnings")
T("hazard", "hazard", ["ocean", "weather", "advisory", "geospatial"], "deterministic risk
decision", True)
T("evidence", "evidence", ["hazard", *deps], "verify claims, detect conflicts, compute confidence")
T("alert", "alert", ["evidence"], "situational alerts")
T("communication", "communication", ["evidence"], "render answer")

```

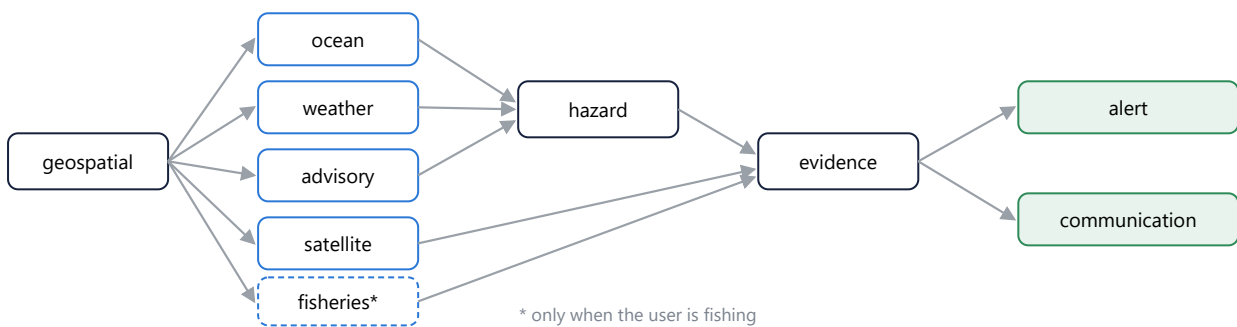


Figure 7 — The safety-check plan as a dependency graph. Five retrieval agents can run at the same time because they all depend only on geospatial.

There are templates for every intent: safety check, conditions, hazard scan, regional risk, route plan, fishing zones, research trend, period comparison, geofence check and follow-ups ("which sources disagree?"). A follow-up needs no new data, so its plan is just `evidence (reuse previous blackboard) → communication`.

Why this matters

The plan is data, not code paths. It is stored with the answer and shown in the UI, and it is what LangGraph executes. When a judge asks "why did the research agent not run for a route question?", the answer is visible in the plan, and the evaluation checks it automatically ("agent routing accuracy").

In ORCA

A plan template is deliberately *not* hard-wired into the LangGraph graph. The graph is generic. It runs whatever dependency graph the planner produced (section 16). That is what lets one compiled graph serve ten different intents.

10. Re-planning: agents that change their mind

A fixed plan is not enough. Sometimes the evidence that comes back says "you asked the wrong question" or "you don't have enough to decide". After each round of work, the planner's `review()` step applies four explicit re-planning rules:

Rule	Trigger	What the planner adds
RP1	A critical retrieval (waves or wind) returned no data	A task for an alternative model (DWD GWAM for waves, UK Met Office for wind), then re-runs hazard → evidence → alert → communication
RP2	Sources disagree on wave height or wind <i>in a decision-relevant way</i> (they fall in different risk levels)	An independent tie-breaker model, then re-verification
RP3	An active tropical cyclone is within 1,500 km	An extended-horizon hazard assessment (+24 h), then refreshed evidence and answer
RP4	The recommended route enters a marine protected area	A re-route with protected areas as a hard constraint. If no such route exists, ORCA says so.

apps/api/orca/agents/planner.py – RP1

```
if oc and oc.status == AgentStatus.FAILED and "RP1-ocean" not in done_flags:
    new.append(PlanTask(id=f"ocean_alt#{round_no + 1}", agent="ocean", round=round_no + 1,
critical=True,
                        reason="primary wave models returned no data – querying alternative model (DWD
GWAM)",
                        params={**base_params, "tiebreaker": True}))
    record("RP1-ocean", "No wave data from primary models → re-planning with an alternative wave model")
```

Two safety rails keep re-planning under control. Each rule fires **at most once** per question (`done_flags`), and there are **at most three rounds** in total (`MAX_ROUNDS = 3`). Each re-plan is recorded with its rule, round and reason, and shown as an orange "RE-PLAN" line in the live trace.

Everyday analogy

A doctor orders blood tests. If one lab result is missing, they order it again from another lab (RP1). If two results contradict each other, they order a third test (RP2). They don't order tests forever. After a couple of rounds they decide with what they have and say what remains uncertain.

11. Evidence, confidence and conflicts

The Evidence agent is where the agents' results become something a person can trust. It works with three objects:

- **Claim:** a statement such as "Significant wave height reaches CAUTION level (worst 2.44 m, onset 08:30 IST)".
- **Evidence item:** one source's support for a claim, with source, product time, retrieval time, location, freshness and a **lineage**. Two products from the same upstream model share a lineage and are not counted as independent.
- **Conflict:** two sources that differ by more than a tolerance (e.g. waves 0.5 m or 25 %). A conflict is *decision-relevant* when the sources fall in different risk levels.

Confidence is **computed, never generated**, in three layers:

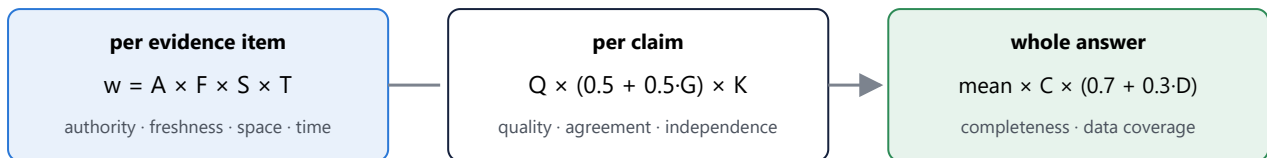


Figure 8 — The confidence chain. Every factor is shown in the UI, with the weakest link named.

A is the authority of the source (national official 0.95 ... ORCA-curated 0.50), **F** its freshness, **S** how close the product's cell is to the point asked about, and **T** how far ahead the forecast is. **G** is how many independent items agree, and **K** rewards having at least two independent lineages. **C** is how many of the expected factors were available, and **D** the share of queried critical sources that answered.

Key idea

Conflicting sources are never averaged. The worst-case value decides, the disagreement lowers confidence through G, and re-planning rule RP2 fetches a tie-breaker. The user sees all three effects.

12. Designing for failure

In LIVE mode, real APIs fail every day: timeouts, rate limits, a model run that is late, an Indian government API that only answers whitelisted IP addresses. We designed each failure to have a defined, visible outcome:

What fails	What ORCA does
One source of several	The agent returns PARTIAL. The source appears under "excluded sources" with its reason. The coverage term D lowers confidence.
A source keeps failing	The transport's circuit breaker opens after 3 failures and stops calling it for 60 s. A cached value up to 24 h old may be used and is labelled "stale-if-error".
An agent crashes or hangs	Its graph node catches the exception or the timeout and records FAILED. Dependent agents still run and handle the missing input.
All wave models fail	RP1 tries an alternative model. If that also fails, risk rule R1 returns NO RELIABLE ASSESSMENT with confidence 0, never "appears safe".
The question is ambiguous	The graph routes to a <code>clarify</code> node and asks, instead of guessing a location.

Watch out

During development, degraded mode (three sources down) still showed the same confidence as the full-data answer, because each remaining claim looked fine on its own. The fix was the coverage term D in the final formula. Failures must show up in the confidence number, not just in a list.

PART 3

Orchestrating with LangGraph

Part 2 described the agents. This part shows how they are wired together into one LangGraph `StateGraph`: how the state is split, how a dynamic plan becomes parallel supersteps, how re-planning loops back, and what a real run looks like.

13. Why LangGraph, and what it replaced

14. The graph state and the runtime context

15. The nodes

16. Dynamic fan-out with Send

17. Conditional edges and the re-plan loop

18. One real run, superstep by superstep

19. Streaming the graph to the screen

20. Testing the graph

13. Why LangGraph, and what it replaced

ORCA's first orchestrator was a few dozen lines of hand-written `asyncio`: a loop that found the tasks whose dependencies were done, started them together with `asyncio.gather`, waited, and repeated. A second loop called the planner's review up to two more times. It worked, but everything was implicit: the order of phases, the loop, the stop condition, the join.

We moved the orchestration to LangGraph for four concrete reasons:

1. **The structure becomes declared.** "understand, then plan or clarify; fan out; join; review; loop or finish" is written as nodes and edges and compiled. It can be printed as a diagram (`GET /api/agents/graph` returns it) and checked by tests.
2. **Parallelism and joins are built in.** LangGraph's superstep model gives exactly the semantics we need: everything that is ready runs concurrently, and the next step waits for all of it.
3. **Dynamic fan-out is a first-class feature.** ORCA's plan differs for every intent. LangGraph's `Send` API lets a routing function start any number of agent nodes with their own payload, which a static graph cannot do.
4. **Loops have guard rails.** Conditional edges express the re-plan loop directly, and the recursion limit guarantees it terminates even if a rule misbehaves.

Just as important is what we did **not** hand to LangGraph. It schedules, but it never decides. The plan, the re-planning rules, the risk model and the confidence model stay in plain, deterministic Python that can be unit-tested without a graph at all.

Why this matters

The migration was checked against behaviour, not taste. The same 83 existing tests passed unchanged, two new graph tests were added, and the full evaluation suite gave identical accuracy, groundedness and hallucination scores before and after (section 26).

14. The graph state and the runtime context

The most important LangGraph design decision was **what goes into the graph state**. We split ORCA's per-question data in two:

```

class OrcaState(TypedDict, total=False):
    completed: Annotated[list[str], operator.add] # task ids, merged across parallel branches
    round: int # re-plan reviews performed
    superstep: int # waves dispatched so far
    clarify: bool

class TaskInput(TypedDict):
    """Payload of a `Send` to an agent node."""
    task_id: str
    wave: int

@dataclass
class OrcaContext:
    run: RunState

```

- **Graph state (`OrcaState`)** holds only *scheduling* facts: which tasks are completed, which re-plan round we are in, how many supersteps ran, and whether clarification is needed. It is small, serialisable, and exactly what the routing functions need to decide the next step.
- **The reducer on `completed` is `operator.add`**. Five agents finishing in the same superstep each return `{"completed": ["ocean"]}`, `{"completed": ["weather"]}` ..., and LangGraph concatenates them instead of letting the last one overwrite the others.
- **Runtime context (`OrcaContext`)** carries the heavy, live objects: the typed Blackboard, the execution context for the current data mode (LIVE, REPLAY or DEMO transport and clock), large in-memory arrays, and the event bus that streams to the browser. LangGraph passes it to every node via `runtime.context` but never copies or serialises it.

Everyday analogy

The graph state is the airport departure board: short, shared, and it tells everyone what happens next. The runtime context is the baggage hall. It is huge and full of real things, and it would be absurd to print it on the board.

Watch out

It's tempting to put the whole Blackboard into the LangGraph state. We didn't. It contains forecast grids and hourly series for many models, and agents append to it concurrently. Keeping it in the runtime context means no copying, and the agents keep the exact contract from Part 2.

15. The nodes

The compiled ORCA graph has **18 nodes** (plus LangGraph's START and END):

Node	Kind	What it does
<code>understand</code>	planner phase	Language, intent, places, time window. Sets <code>clarify</code> if something essential is missing.
<code>clarify</code>	planner phase	Builds a "clarification needed" answer instead of guessing.
<code>plan</code>	planner phase	Builds the <code>ExecutionPlan</code> for the intent and registers every task on the blackboard.
<code>geospatial ... alert</code> (12)	agent	One node per specialised agent. Receives a <code>TaskInput</code> , runs that agent's task with timeout and failure isolation, and returns <code>{"completed": [task_id]}</code> .
<code>join</code>	barrier	Runs once after every parallel superstep and counts the wave.
<code>review</code>	planner phase	Applies re-planning rules RP1–RP4 and appends new tasks to the plan.
<code>visualize</code>	output	Builds the map directive: layers, markers, route and zone geometry, camera position.

All twelve agent nodes are created by one small factory, so each agent is a real, separately named node in the graph while sharing the same failure handling:

`apps/api/orca/graph.py`

```
def make_agent_node(name: str):
    async def node(payload: TaskInput, runtime: Runtime[OrcaContext]) -> OrcaState:
        st = runtime.context.run
        task = next(t for t in st.bb.plan.tasks if t.id == payload["task_id"])
        await execute_task(task, st, payload.get("wave", 0))
        return {"completed": [task.id]}
    node.__name__ = f"{name}_node"
    return node
```

`execute_task` is where the agent contract from section 7 meets the graph. It marks the record `RUNNING`, emits an `agent_start` event, calls `agent.run()` under `asyncio.wait_for(..., timeout=agent.timeout_s)`, converts any exception or timeout into a `FAILED` outcome, copies status, tools, sources, failures and the typed output onto the blackboard, and emits `agent_end`. Because it never raises, one broken agent can never stop the graph.

16. Dynamic fan-out with Send

This is the central trick. The graph doesn't contain edges like "ocean → hazard", because those edges differ between intents. Instead, a routing function reads the **plan's** dependency graph at run time and returns one `Send` per task that is ready:

```
def _ready(st: RunState, completed: set[str]) -> list[PlanTask]:
    """Tasks of the plan whose dependencies are all satisfied."""
    ids = {t.id for t in st.bb.plan.tasks}
    pending = [t for t in st.bb.plan.tasks if t.id not in completed]
    ready = [t for t in pending if all(d in completed or d not in ids for d in t.depends_on)]
    if pending and not ready: # dependency cycle / unknown dependency – run everything left
        ready = pending
    return ready

def dispatch(state: OrcaState, runtime: Runtime[OrcaContext]):
    """Fan out every ready task in parallel, or hand over to the planner's review when the plan is
    done."""
    st = runtime.context.run
    if not _ready(st, set(state.get("completed", []))):
        return "review"
    return _sends(st, state) # [Send(t.agent, {"task_id": t.id, "wave": n}) for t in ready]
```

Put this together with LangGraph's superstep rule and you get a full parallel DAG executor in a few lines:

1. `dispatch` returns, say, five `Send` s: ocean, weather, satellite, advisory, fisheries.
2. LangGraph runs those five agent nodes **concurrently** in one superstep. The network calls overlap.
3. Every agent node has a static edge to `join` . LangGraph runs `join` **once**, after all five finished, and the reducer has merged their five `completed` updates.
4. `join` 's conditional edge calls `dispatch` again, which now finds hazard ready. Repeat.

Key idea

One LangGraph superstep = one "wave" of the plan: every task whose dependencies are satisfied. The planner decides *what* depends on what. LangGraph decides *when* things run, and runs independent things together.

In ORCA

The same agent can receive two `Send` s in one run, e.g. the normal `ocean` task and later `ocean_tiebreak#1` . Each `Send` carries its own `task_id` , so the node knows which parameters (such as `tiebreaker: True`) to use.

17. Conditional edges and the re-plan loop

Here is the complete graph builder. It is the most important 25 lines in the project:

```
def build_graph():
    g = StateGraph(OrcaState, context_schema=OrcaContext)
    g.add_node("understand", understand)
    g.add_node("clarify", clarify)
    g.add_node("plan", plan)
    for name in AGENTS:
        g.add_node(name, make_agent_node(name), input_schema=TaskInput)
        g.add_edge(name, "join")
    g.add_node("join", join)
    g.add_node("review", review)
    g.add_node("visualize", visualize)

    g.add_edge(START, "understand")
    g.add_conditional_edges("understand", after_understand, ["clarify", "plan"])
    g.add_edge("clarify", "visualize")
    agents = list(AGENTS)
    g.add_conditional_edges("plan", dispatch, agents + ["review"])
    g.add_conditional_edges("join", dispatch, agents + ["review"])
    g.add_conditional_edges("review", after_review, agents + ["visualize"])
    g.add_edge("visualize", END)
    return g.compile(name="orca")
```

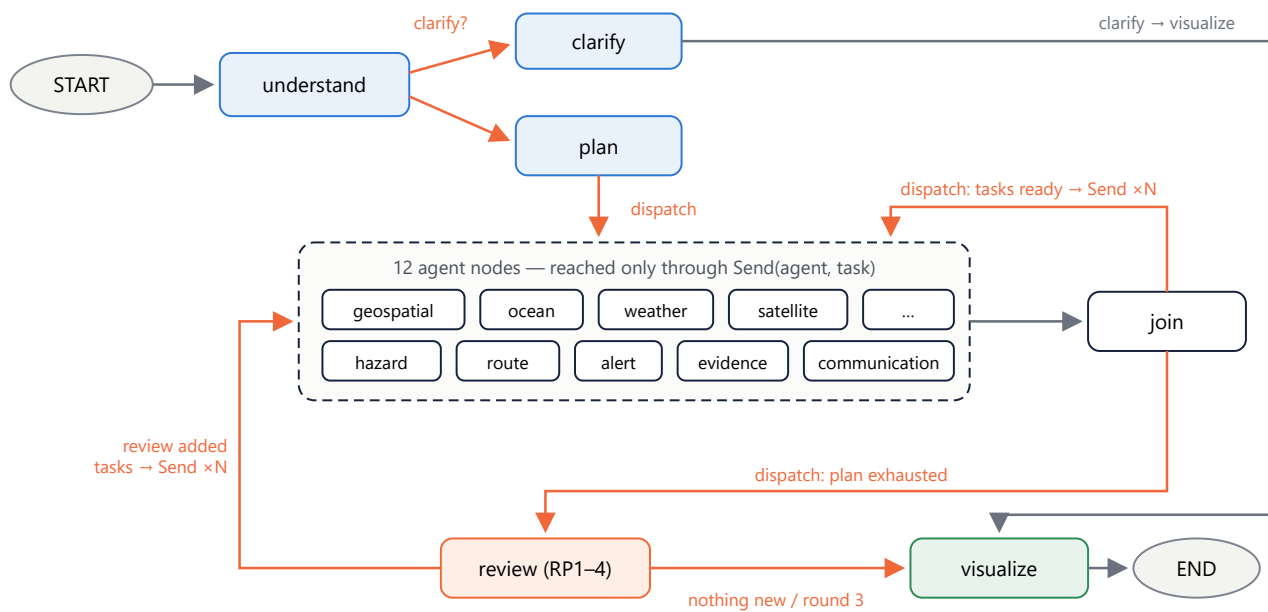


Figure 9 — The compiled ORCA graph. Grey: fixed edges. Orange: conditional edges whose destination is decided at run time.

The three routing functions are tiny, and that is the point:

- `after_understand` returns `"clarify"` or `"plan"`.
- `dispatch` (after `plan` and after every `join`) returns Sends for ready tasks, or `"review"` when the plan is exhausted.
- `after_review` returns Sends if review added tasks and the round cap (3) is not reached, otherwise `"visualize"`.

Re-planning therefore needs no special machinery. `review` appends tasks to the plan, and the *same* `Send` fan-out runs them. A tie-breaker task depends on nothing new, so it starts immediately. The re-run of

hazard depends on the tie-breaker, so it waits for it. LangGraph's recursion limit (set to 200 supersteps, while a heavy route question uses about 25) is a final guarantee that no bug can loop forever.

apps/api/orca/orchestrator.py

```
await GRAPH.ainvoke({}, config={"recursion_limit": RECURSION_LIMIT, "run_name": bb.trace_id},
                    context=OrcaContext(run=st))
```

18. One real run, superstep by superstep

Let's follow one question through the graph, using DEMO scenario `conflicting_sources`, where the wave and wind models disagree on purpose:

"Is it safe to fish 25 km off Chennai tomorrow morning?"

`understand` resolves: intent SAFETY_CHECK (rule-based, confidence 0.95), place Chennai, 25 km offshore, vessel small craft, window "Tomorrow 05:00–11:00 IST" with the recorded assumption '*morning*' interpreted as 05:00–11:00 IST. `plan` builds 10 tasks for 10 agents. Then:

Superstep	Round	Tasks (in parallel)	What happened
1	0	geospatial	7 sample points, 24.1 km from the coast, depth 49 m, 262 km to the Sri Lanka boundary (453 ms)
2	0	ocean, weather, satellite, advisory, fisheries	Waves worst 2.44 m across 3 models, spread 1.14 m. Wind worst 31 km/h. Satellite SST 29.87 °C. 1 official warning (simulated INCOIS). All five overlap in time (≈ 95–367 ms each).
3	0	hazard	CAUTION, risk index 91. Drivers: wind, gusts, wave height, official advisory.
4	0	evidence	15 claims, 25 evidence items, 4 conflicts , confidence 57 %
5	0	alert, communication	4 alerts generated, 2 verified. Answer rendered.
review: RP2 fires twice. Wind conflict (GFS vs ECMWF) and wave conflict (MFWAM vs others) are decision-relevant, so tie-breakers are requested.			
6	1	weather_tiebreak#1, ocean_tiebreak#1	UK Met Office and DWD GWAM added. Now 4 models each (≈ 145 ms, in parallel).
7–10	1	hazard#1 → evidence#1 → alert#1 → communication#1	Still CAUTION. 29 evidence items, confidence 56 %. The tie-breakers did not settle the disagreement, so confidence stays moderate.

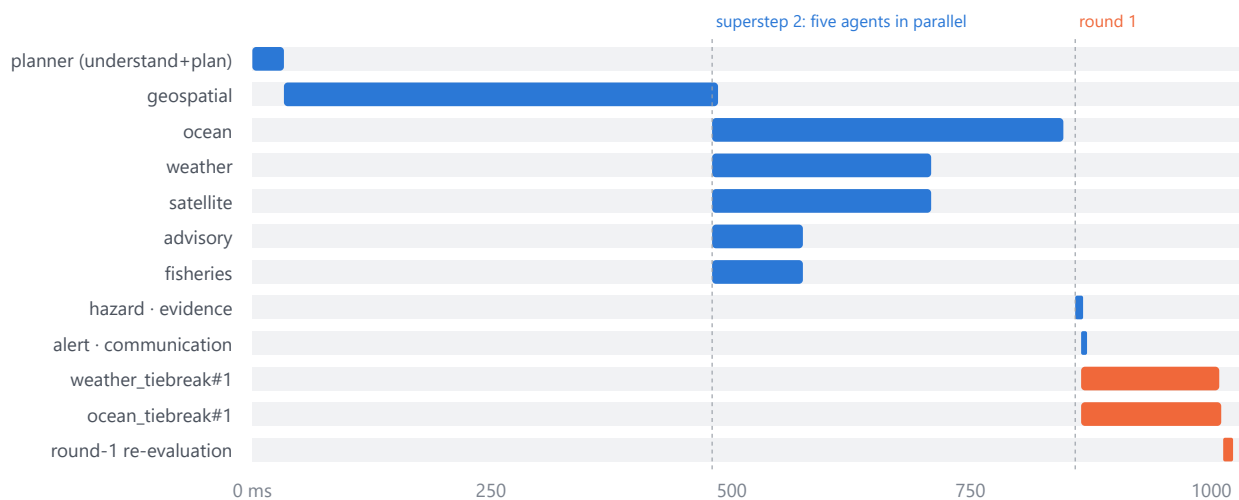


Figure 10 — Real timings from this run (DEMO mode, 1.02 s total). Blue: round 0. Orange: tie-breakers from re-planning round 1. Sub-millisecond agents are drawn as ticks.

The final answer: **CAUTION — hazardous conditions possible**, confidence 56 %. The four top reasons each name the worst value, the source that produced it, the thresholds and the spread, e.g. "Significant wave height: worst value 2.4 m (Météo-France MFWAM); caution ≥ 1.5 , danger ≥ 2.5 m; 4 sources, spread 1.14 m — conservative (worst-case) value used".

Key idea

Superstep 2 is where the multi-agent design pays off. Five independent experts fetch from about a dozen sources at the same time, so the slowest of them sets the wall-clock time, not the sum. In this run the five retrievals used 1,013 ms of agent time but only 367 ms of wall-clock time. In LIVE mode, where each upstream call takes hundreds of milliseconds, the saving grows accordingly.

19. Streaming the graph to the screen

A judge or a fisherman should be able to *watch* the agents work. Every node emits typed **trace events** on the run's event bus, which the API streams to the browser as Server-Sent Events (SSE, a simple one-way stream over HTTP):

Event	Emitted by
start	the orchestrator, with the mode and <code>engine: "langgraph"</code>
stage	agents, as they reach the 16 reasoning stages (INTENT ... RESPONSE)
plan	<code>plan</code> and <code>review</code> , with the full task list
dispatch	the <code>dispatch</code> router, once per superstep: which tasks start in parallel
agent_start / agent_end	every agent node: reason, status, duration, sources, failures
tool	connectors: each source call with its freshness
replan , conflict , alert	review, evidence, alert
done	the orchestrator, with the decision and total time

The waves are also stored permanently on the blackboard as `supersteps`, a list of `{wave, round, tasks, agents}`. The console's **Agents** page shows them under the live agent network, and the **Evidence** page replays the whole event log for any past trace ID (e.g. `ORCA-2026-09-27-7F3A1`).

In ORCA

The compiled topology itself is available at `GET /api/agents/graph`: nodes, edges with a `conditional` flag, and LangGraph's own Mermaid drawing of the graph, generated by `GRAPH.get_graph().draw_mermaid()`.

20. Testing the graph

ORCA's agents and core models were already covered by tests before the LangGraph migration. We added two tests for the graph itself:

apps/api/tests/test_pipeline.py

```

def test_langgraph_topology():
    d = graph_description()
    assert d["engine"] == "langgraph"
    ids = {n["id"] for n in d["nodes"]}
    assert set(AGENTS) <= ids and {"understand", "plan", "join", "review", "visualize", "clarify"} <=
ids
    edges = {(e["source"], e["target"]) for e in d["edges"]}
    assert all((a, "join") in edges for a in AGENTS) # every agent fans back into the barrier

async def test_supersteps_are_parallel_waves():
    bb = await run("conflicting_sources")
    waves = bb.supersteps
    assert waves[0]["tasks"] == ["geospatial"]
    assert {"ocean", "weather", "advisory"} <= set(waves[1]["tasks"]) # independent agents share a
superstep
    done = set()
    for w in waves: # a task only runs after its
dependencies
        for tid in w["tasks"]:
            t = next(t for t in bb.plan.tasks if t.id == tid)
            assert all(d in done for d in t.depends_on)
            done |= set(w["tasks"])
    assert any(w["round"] == 1 for w in waves) # RP2 re-plan executed as further
supersteps

```

The end-to-end tests run every DEMO scenario through the compiled graph. They check, for example, that the cyclone scenario fires RP3 and returns DON'T GO, and that a total outage returns NO RELIABLE ASSESSMENT with confidence 0. All run without any network or API key.

Why this matters

Testing without an AI and without the internet makes the tests fast (the full suite takes about 22 seconds), free, and deterministic. A test that sometimes fails because an API was slow teaches you nothing.

PART 4

The rest of the system

Agents need data to fetch, models to reason with, an API to be called through, and a screen to be seen on. This part tours everything around the agent graph, then tells how the project was built.

21. The big picture

22. Data connectors and the three modes

23. The deterministic core

24. Where the LLM is allowed, and its guard

25. The API and the mission-control console

26. Evaluation: how we know it works

27. How it was built

21. The big picture

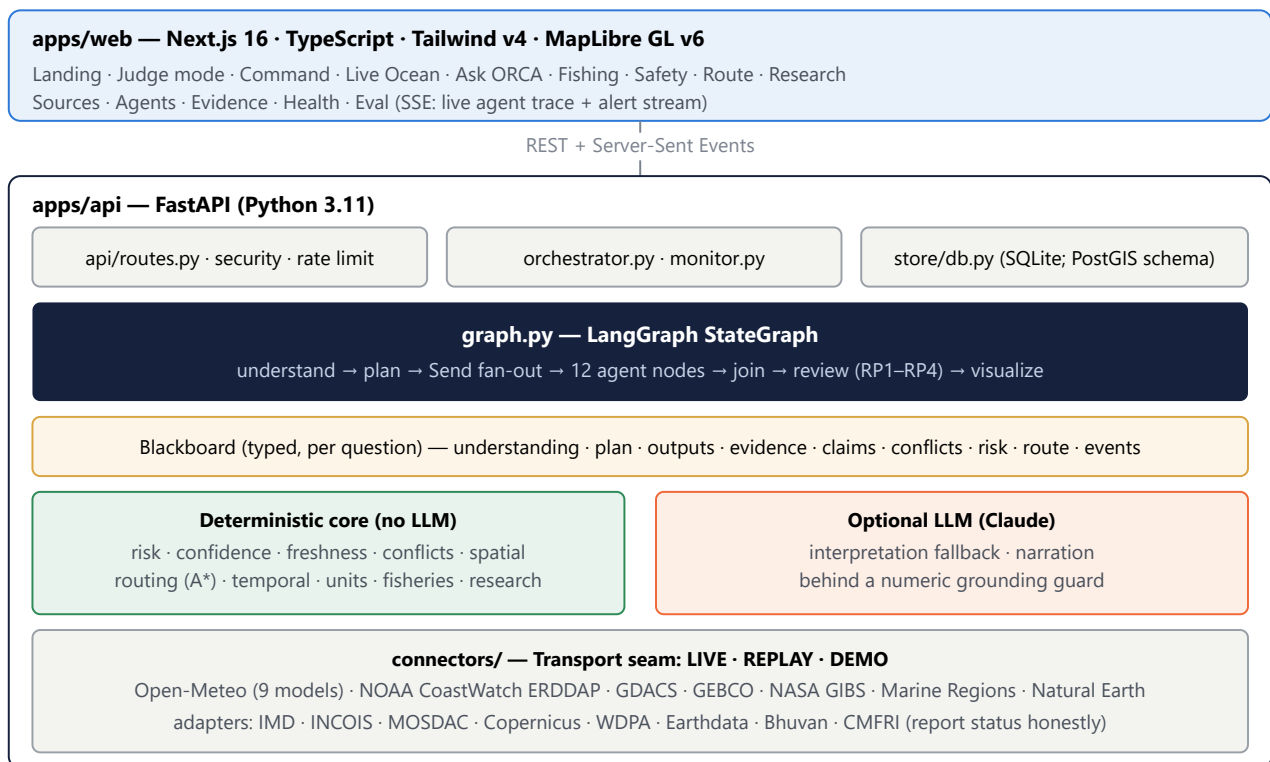


Figure 11 — ORCA's architecture. The LangGraph agent graph sits in the middle, with the typed blackboard directly below it.

A request goes: browser → `POST /api/query` → the orchestrator creates a blackboard and an execution context for the chosen mode → the compiled graph runs → events stream back live → the finished blackboard is stored in SQLite under a trace ID → the browser renders the decision, the evidence and the map.

22. Data connectors and the three modes

A **connector** knows how to ask one data provider a question and how to parse its answer into ORCA's typed series with provenance. Underneath every connector sits a **transport** that decides where the bytes actually come from. This one seam is what makes ORCA's three modes possible:

Mode	Transport	Behaviour
LIVE	LiveTransport (httpx)	Real APIs only. Per-source TTL cache, request coalescing, 2 retries with back-off, circuit breaker (opens after 3 failures, retries after 60 s), stale-if-error up to 24 h (labelled), optional recorder.
REPLAY	ReplayTransport	Serves responses recorded from real APIs (the bundled snapshot has 85). The clock is frozen at recording time. An unknown request fails with <code>NOT_IN_RECORDING</code> and is never invented.
DEMO	DemoTransport	A synthetic but physically motivated ocean produces responses <i>in the exact upstream formats</i> (Open-Meteo JSON, ERDDAP tables, GDACS GeoJSON), including scripted outages. Everything is labelled <code>SIMULATED</code> , and a striped banner is shown.

Key idea

The agents are identical in all three modes. They don't know whether bytes came from the internet, a recording or a simulator. That is why DEMO is a true test of the real agents and not a mock-up, and why the evaluation can run offline.

The restricted Indian sources (IMD, INCOIS, MOSDAC and others) have adapters that activate on configuration. Today they report their real status, for example IMD's "*IP needs to be whitelisted*", and every safety answer tells the user to check official IMD/INCOIS bulletins.

23. The deterministic core

The reasoning agents are thin. They gather inputs and call pure functions in `apps/api/orca/core/` and `reasoning/` that have no network access and are unit-tested directly.

The risk model `orca-risk-1.0`

For each factor it takes each source's worst value across all sample points and hours, compares it with vessel-specific thresholds (e.g. small craft: waves caution 1.5 m / danger 2.5 m, wind 28 / 45 km/h), and applies ordered rules:

Rule	Effect
R1	A critical factor (wind or waves) is unavailable → NO RELIABLE ASSESSMENT
R2	Any factor at DANGER → DON'T GO. R2b : thunderstorm potential is model-inferred, so it can raise the decision to CAUTION but never alone to DON'T GO.
R3	All sources of a critical factor are STALE → at least CAUTION
R4	Any factor at CAUTION → CAUTION
R5	Sources straddle the caution threshold on a critical factor → at least CAUTION
R6	Otherwise GO: "within configured limits, not a guarantee of safety"

Thunderstorm energy (CAPE) only counts where the same model also predicts at least 0.5 mm/h of rain at the same place and hour. Without that gate, ORCA returned DON'T GO on calm, dry days when tested on live data. That is how the rule came about.

Routing

A time-dependent A* search over a land-masked grid. The cost of each step is `distance × (1 +  $\alpha \cdot \text{risk}(\text{place}, \text{ETA})$ )`, with $\alpha = 0$ for the shortest route, 3 for the recommended route and 8 for the safest. Restricted zones (and foreign EEZs for fishing craft) are impassable, and protected areas cost six times more unless RP4 blocks them outright.

Freshness, conflicts, geometry, statistics

Freshness uses two clocks per datum (product age and retrieval age). Conflicts use per-variable tolerances. Geometry uses shapely with local azimuthal-equidistant projections for exact kilometres. Research uses Welch's t-test, least-squares trends and lagged Pearson correlation. All parameters are published in `docs/REASONING.md`.

24. Where the LLM is allowed, and its guard

ORCA works fully without any LLM. If an Anthropic API key is configured, Claude (`claude-opus-5`, with server-side fallback models enabled) is used in two narrow places:

1. **Interpretation fallback.** When the rule-based parser's confidence is below 0.5, Claude returns a JSON object that must match a schema and must name places by *gazetteer identifiers*. Anything that fails validation is discarded.
2. **Narration.** Claude may rephrase the verified result more naturally. The **grounding guard** then extracts every number from the text and checks that each one appears, exactly or as a rounding, in the facts it was given. One unsupported number and the narration is thrown away and templates are used.

Watch out

The guard once rejected perfectly good narrations, because the facts were serialised as JSON with escaped characters (`-` instead of `"-"`), so the model's numbers didn't match the text they were compared with. The fix was `json.dumps(..., ensure_ascii=False)`. Guards need tests for false alarms too, not only for catching lies.

25. The API and the mission-control console

The backend is **FastAPI**. The main endpoints:

Endpoint	Purpose
POST /api/query	Ask a question. Returns a query ID, or waits for the answer with <code>wait=true</code> .
GET /api/query/{id}/events	Live SSE stream of that run's trace events
GET /api/query/{id}	The full stored blackboard (by query or trace ID)
GET /api/agents · /api/agents/graph	Agent catalogue with output schemas · compiled LangGraph topology
POST /api/route/analyze	Route optimisation and segment risk
GET /api/sources · /api/system/health	Source status and freshness · agents, queries, cache, monitor, LLM usage
GET /api/alerts/stream	Live SSE stream of new alerts
POST /api/eval/run	Run the evaluation suite (admin)

Security is prototype-grade but deliberate: bearer-token roles, admin-only evaluation, token-bucket rate limits, validated inputs, secrets only from environment variables (and stripped from recordings), and stored locations rounded to 0.1°.

The frontend is a **Next.js 16** "mission control" console with a full-screen **MapLibre** map. The basemap is built from bundled Natural Earth land data, so no API keys or tile licences are needed, and real NASA GIBS satellite imagery can be overlaid. The agent activity dock at the bottom streams the live trace. The pages that matter most for agent design are:

- **Agents:** the live agent network lighting up node by node, each agent's tools, inputs and typed output schema, the LangGraph supersteps, and the re-planning log.
- **Evidence:** every claim with its evidence items, conflicts, excluded sources, the full confidence computation and the complete event log for any trace ID.
- **Judge mode:** a guided tour through the DEMO scenarios.

26. Evaluation: how we know it works

The evaluation suite (`orca-eval-v1`) has **100 understanding cases** (English safety questions for ten harbours, questions in seven Indian languages, and fishing, route, regional, research, comparison, hazard, geofence and follow-up questions) and **14 end-to-end cases** run through the full graph in DEMO mode. Results after the move to LangGraph, 27 September 2026:

1.00 intent accuracy (100)	1.00 language detection	1.00 decision accuracy (14)	1.00 groundedness
0.00 hallucination rate	1.00 agent routing accuracy	1.00 tool selection accuracy	1.00 risk consistency

0.96

evidence completeness

0.79 / 5.0 s

latency p50 / p95 (DEMO)

Agent routing accuracy checks that the required agents ran and irrelevant ones didn't. **Tool selection** checks that the required tools appear in the trace. **Risk consistency** checks that paraphrases and translations of the same question (e.g. Kochi in English, Malayalam and Hindi) get the same decision, and that a re-run is identical. **Groundedness** checks that every number in an answer exists in the computed facts.

Watch out

The dataset was written by the same team that wrote the rules, so perfect scores show **regression safety, not generalisation**. The honest next step is an independent test set in fishermen's and officials' own words, and checking past decisions against what the sea actually did.

27. How it was built

ORCA was built in layers, from the contract outwards, so each layer could be tested before the next one depended on it:

1. **Schemas first.** Provenance, freshness, evidence, claims, assessment, blackboard: the shared vocabulary every agent speaks, written as Pydantic models before any agent existed.
2. **The deterministic core.** Freshness rules, units, geodesy, the risk model, confidence and A* routing, each with unit tests and no network.
3. **Connectors and the transport seam.** Real APIs first (Open-Meteo, ERDDAP, GDACS, GEBCO, GIBS), then the recorder, REPLAY and the DEMO simulator speaking the same formats.
4. **The agents**, one at a time, each against its contract: retrieval agents, then geospatial, hazard, evidence, communication, route, fisheries, research and alert.
5. **The planner and re-planning rules**, with plan templates per intent and RP1–RP4.
6. **Orchestration.** First a hand-written asyncio DAG scheduler, then migrated to a LangGraph `StateGraph` with one node per agent, `Send` fan-out and conditional re-planning edges, verified with the unchanged test suite and evaluation.
7. **The console**, evaluation suite, documentation and Docker packaging.

Bugs that shaped the design

What went wrong	What we changed
Coarse models (0.25–0.5° grids) were penalised as if they were far away	The spatial factor S now allows for native resolution: $\exp(-\max(0, d - 0.71r)/50)$
Model CAPE produced DON'T GO on dry days in LIVE data	CAPE gated by same-model rain, and R2b caps it at CAUTION
The ERDDAP time-series request returned HTTP 404	Region series request the last N days with a server-side stride
Research evaluated hypotheses even with no data	Hypotheses without data are skipped, and a premise check flags questions whose assumption contradicts the data
A route whose end-point lies inside a protected zone cannot avoid it, so the RP4 re-route had nothing to offer	RP4 now reports explicitly when no compliant route exists
The fishing-zone demo took 14.8 s	Vectorised distance-to-coast in the simulator: 1.2 s

PART 5

Using it, and reference

How to run ORCA, a script for demonstrating it, what it can't do yet, and a glossary and code map to keep next to you while reading the code.

28. Setting it up

29. A guided demonstration

30. Limitations

31. Future improvements

32. Glossary

33. Code map and command cheat sheet

34. What you now understand

28. Setting it up

You need Python 3.11+ and Node.js 20.9+. No API keys are required: all LIVE sources used by default are public, and DEMO and REPLAY work offline.

```
python -m venv .venv
.venv/Scripts/pip install -r apps/api/requirements-dev.txt      # Linux/macOS: .venv/bin/pip
.venv/Scripts/python -m uvicorn orca.main:app --app-dir apps/api --port 8000
cd apps/web && npm install && npm run dev
```

Open `http://localhost:3000` and choose **Judge mode** or **Ask ORCA**. The API documentation is at `http://localhost:8000/docs`. With Docker: `docker compose up --build`.

Optional setting	Effect
<code>ANTHROPIC_API_KEY</code>	Enables the LLM interpretation fallback and guarded narration
<code>ORCA_API_TOKENS</code> , <code>ORCA_AUTH_REQUIRED</code>	Bearer-token roles, mandatory authentication
<code>ORCA_MONITOR_INTERVAL_S</code>	How often the background monitor re-checks watched places
IMD / MOSDAC / Copernicus credentials	Activate the restricted-source adapters (see <code>docs/DATA_SOURCES.md</code>)

29. A guided demonstration

The nine DEMO scenarios are designed so that each one exercises a different part of the agent design:

Scenario	Question	What it shows about the agents
kochi_fishing	Is it safe for a fishing boat to leave Kochi tomorrow at 5 AM?	CAUTION with onset $\approx$ 08:30 IST $\rightarrow$ "be back before then". Five retrieval agents in one superstep. The Malayalam version gives the same decision.
approaching_cyclone	Is it safe to fish off Visakhapatnam tomorrow morning?	DON'T GO inside a wind radius. RP3 extends the horizon by 24 h. Verified alerts.
mumbai_goa_route	Find a lower-risk route from Mumbai to Goa for a trawler	Shortest vs recommended route. Segment risk at ETA. RP2 tie-breaker for waves.
pfz_mangaluru	Show potential fishing zones near Mangaluru	SST fronts $\times$ chlorophyll. ORCA-derived indicator kept separate from INCOIS PFZ.
protected_geofence	Plan a route from Thoothukudi to Rameswaram ...	Enters Gulf of Mannar park after 7.5 km. RP4 hard-constraint re-route proves no compliant route exists.
conflicting_sources	Is it safe to fish 25 km off Chennai tomorrow morning?	4 conflicts. RP2 $\times$ 2 tie-breakers in parallel (section 18).
degraded_sources	Is it safe to fish 20 km off Kochi tomorrow morning?	Three sources down: PARTIAL agents, exclusions listed, lower confidence
total_outage	(same question)	All wave models down. RP1 alternative also down $\rightarrow$ NO RELIABLE ASSESSMENT
research_ch1	Why did chlorophyll decline off Kochi over the past 14 days?	Observation / correlation / hypothesis / conclusion. No causal claims.

A good five-minute demonstration: run `conflicting_sources` with the **Agents** page open, so the audience sees the supersteps and the RP2 re-plan happen, then open **Evidence** for the same trace to show the conflicts and the confidence computation. Finish with `total_outage` to show that ORCA refuses rather than guessing.

30. Limitations

- **No live Indian official data yet.** IMD needs IP whitelisting. INCOIS has no public machine API. MOSDAC and Copernicus need accounts. Safety answers rely on international models and GDACS, and say so.
- **No in-situ observations** (buoys, AIS, ship reports), so there is no real-time ground truth. Lightning is inferred, not observed. Storm surge, tides and tsunamis are not modelled.
- **Thresholds are ORCA defaults**, informed by IMD practice but not validated by any authority. The worst-case policy over-warns when one model is an outlier.
- **Confidence is a transparent heuristic**, not a calibrated probability.
- **Zone outlines are approximations** and seasonal ban dates must be checked each year. The gazetteer has 64 places.
- **Indian-language templates were machine-authored** and need review by native speakers.

- **Single process:** in-memory caches and circuit breakers, SQLite, and the LangGraph graph runs without a persistent checkpoint, so an interrupted run is not resumed.

31. Future improvements

Idea	Why
LangGraph checkpointer (e.g. Postgres)	Resume interrupted runs, and "time-travel" through a past run's supersteps for debugging
Human-in-the-loop interrupts	LangGraph's <code>interrupt()</code> could pause before an alert is broadcast so a duty officer can approve it
Per-node retry and cache policies	Declarative retries per agent node instead of only in the transport
Subgraphs per intent	Route planning and research could become their own compiled subgraphs with private state
IMD / INCOIS / MOSDAC integration	Official Indian warnings as first-class evidence
Independent evaluation set, retrospective validation	Measure generalisation, not only regression
PostGIS, Redis, a job queue	Multi-process deployment and shared caches

32. Glossary

Term	Meaning
Agent	A program that plans or performs one kind of work with tools and returns a typed result. In ORCA: a class with name, tools, consumes, output model, timeout and <code>run()</code> .
Blackboard	Shared, typed per-question data structure that agents read from and write to instead of messaging each other
CAPE	Convective available potential energy: an index of thunderstorm potential
Claim / evidence item	A statement ORCA makes / one source's support for it, with provenance and weight
Conditional edge	LangGraph edge whose destination is chosen at run time by a routing function
Confidence	ORCA's computed trust score: A·F·S·T per item, Q·(0.5+0.5G)·K per claim, × C·(0.7+0.3D)
DAG	Directed acyclic graph: tasks with "depends on" arrows and no cycles
EEZ	Exclusive economic zone, a country's sea area out to 200 nautical miles
Freshness	LIVE / RECENT / STALE / UNAVAILABLE, plus REPLAY / SIMULATED / STATIC labels
Gazetteer	ORCA's list of known places with coordinates and local-language names
Grounding guard	Check that every number in LLM text exists in computed facts
LangGraph	Open-source library for building stateful agent workflows as graphs

Lineage	The upstream origin of a datum. Two items with the same lineage are not independent.
MPA	Marine protected area
PFZ	Potential Fishing Zone, INCOIS's official advisory
Reducer	Function that merges state updates to the same key (ORCA: <code>operator.add</code>)
Re-planning (RP1–RP4)	Rules that add tasks after reviewing results
Runtime context	Per-run objects LangGraph passes to nodes outside the state (ORCA: <code>OrcaContext.run</code>)
Send	LangGraph instruction to run a node with a specific payload, used for parallel fan-out
SSE	Server-Sent Events: a one-way live stream from server to browser
SST	Sea-surface temperature
Superstep	One LangGraph tick in which all scheduled nodes run concurrently. In ORCA, one wave of the plan.
Trace ID	ID of one answered question, e.g. <code>ORCA-2026-09-27-7F3A1</code>
Transport	Layer under the connectors that fetches bytes: live, replayed or simulated

33. Code map and command cheat sheet

Path	What lives there
<code>apps/api/orca/graph.py</code>	The LangGraph StateGraph: state, nodes, <code>Send</code> dispatch, re-plan loop, map directive
<code>apps/api/orca/orchestrator.py</code>	Creates a run, invokes the graph, persists the trace, agent catalogue
<code>apps/api/orca/agents/base.py</code>	The agent contract: <code>Agent</code> , <code>AgentOutcome</code> , <code>RunState</code>
<code>apps/api/orca/agents/planner.py</code>	Understanding, plan templates, re-planning rules RP1–RP4
<code>apps/api/orca/agents/*.py</code>	The twelve specialised agents
<code>apps/api/orca/core/</code>	Schemas, risk, confidence, freshness, routing, spatial, temporal, units, clock
<code>apps/api/orca/reasoning/</code>	NLU and lexicon, conflicts, fisheries, research statistics, i18n, LLM layer
<code>apps/api/orca/connectors/</code>	Transports and data-source connectors
<code>apps/api/orca/demo/</code> , <code>evaluation/</code>	DEMO simulator and scenarios · evaluation dataset and runner
<code>apps/api/tests/</code>	85 pytest tests
<code>apps/web/src/app/(console)/</code>	Console pages: agents, evidence, route, research, judge ...
<code>docs/</code>	ARCHITECTURE, AGENTS, REASONING, DATA_SOURCES, SAFETY, API, DEMO, EVALUATION, LIMITATIONS

Command	Does
<code>cd apps/api && ../../.venv/Scripts/python -m pytest -q</code>	Run all tests ($\approx$ 22 s, offline)
<code>curl localhost:8000/api/agents/graph</code>	Print the compiled LangGraph topology
<code>curl -X POST "localhost:8000/api/query?mode=DEMO&scenario=conflicting_sources&wait=true" -H "content-type: application/json" -d '{"text\":"Is it safe to fish 25 km off Chennai tomorrow morning?\"}"</code>	Ask a question and wait for the full blackboard
<code>python scripts/record_replay.py</code>	Record a new REPLAY snapshot from live sources
<code>python scripts/fetch_reference_data.py</code>	Refresh bundled reference geodata
<code>docker compose up --build</code>	Run API and web in containers

34. What you now understand

- Why a marine safety answer needs many sources, and why they must never be averaged or invented.
- What an agent is, and why "agentic" does not have to mean "an LLM decides everything".
- How thirteen specialised agents cooperate through a typed blackboard instead of free-text messages.
- The agent contract (tools, consumes, typed output, timeout, `run()`) and why each part of it exists.
- How the planner turns a sentence in eight languages into a dependency graph of tasks, and how rules RP1–RP4 change the plan when the evidence demands it.
- How evidence, conflicts and a computed confidence make an answer auditable.
- How LangGraph runs it all: a small scheduling state with a reducer, a runtime context for the heavy data, one node per agent, `Send` fan-out into parallel supersteps, a join barrier, and conditional edges that loop back for re-planning.
- How LIVE, REPLAY and DEMO run the same agents, and how the evaluation checks routing, grounding and consistency.

Key idea

In ORCA the agents are experts with contracts, the planner is the strategist, LangGraph is the conductor, and the evidence agent is the referee. None of them is allowed to make up a number.